

Cours Programmation Orientée Objet : JAVA 1

Enseignant : **Malek BEN SALEM**

mal.bensalem@gmail.com

3^{ème} année Génie Informatique

PRÉSENTATION DU COURS

❑ Objectif général :

Ce cours vise à faire acquérir à l'apprenant :

- Le principe et les concepts de la programmation Orienté Objet,
- La syntaxe de base du langage Java
- Le concept orienté objet avec Java
- Maîtriser Les notions de paquetages, d'héritage, des classes anonymes, des classes abstraites, des interfaces et des exceptions.
- Maîtriser la programmation Orienté Objet avec la pratique

Références du Cours

- **Livre** « Java : Les fondements du langage » de Thierry RICHARD, édité par **Eni Editions** en **Mars 2022**.
- **Livre** « Apprendre la programmation Orienté Objet avec le langage Java » de Luc GERVAIS, édité par **Eni Editions** en **Août 2020**.
- **Livre** « Java et Eclipse : Développez une application avec Java et Eclipse (2e édition) » de Frédéric DÉLÉCHAMP, édité par **Eni Editions** en **Mars 2018**.

Plan du Cours

1. **Chapitre 1 : Généralité et syntaxe de base**
2. Chapitre 2 : Concepts Objets en java
3. Chapitre 3 : Héritage
4. Chapitre 4 : Classes abstraites et Interfaces
5. Chapitre 5 : Les exceptions

Chapitre 1: Généralité et syntaxe de base

1. Introduction à JAVA
2. Mise en œuvre
3. Java par l'exemple
4. Structure d'une application JAVA standard
5. Ecrire une application Java avec un IDE
6. Syntaxe de base de java
7. Références Utiles

1. Introduction à Java

- **Un langage jeune**

- Né depuis les années 1990
- Présenté officiellement en 1995 par SUN
- Détenu par ORACLE depuis 2009 après le rachat de SUN par ORACLE.
- En pleine évolution et l'on a vu passer les versions successives 1.01, 1.02, 1.1, 1.2 (notée Java 2) et aujourd'hui JDK 25 (Septembre 2025).
- Versions JDK LTS (Long Term Support) : JDK 8, 11, 17, 21 et 25.

- **L'argument principal de Java**

- C et le C++ sont très **fortement dépendants** du **processeur**,
- ☹ Pour qu'un programme fonctionne sur plusieurs systèmes différents, il faut le compiler plusieurs fois (parfois des modifications partielles de code).
- ☺ Idée Java : écrire le même programme, quelque soit le système support.

1. Introduction à Java

- **Le langage Java est « familier » :**
 - Java est un langage très proche du langage C, C++. Par exemple :
 - Même types de base que C++ (int, float, double, etc.),
 - Même formes de déclarations que C++,
 - Même structure de contrôle que C++ (if, while, for, etc.).
- **Le langage Java est simple :**
 - Java est un langage simple par rapport au langage C et C++.
 - Il n'y a plus de pointeurs et des manipulations les concernant ;
 - Java se charge (presque) de restituer au système les zones mémoire inaccessibles et ce sans l'intervention du programmeur.

1. Introduction à Java

- **Le langage Java est orienté objet :**
 - Paquetage pour la réutilisation :
 - java.lang: classes de base
 - java.awt : interfaces graphiques
 - java.net : communication réseaux (socket et URL)
 - Java.applet: API Applet
- **Le langage Java est distribué :**
 - Supporte des applications réseaux (protocoles de communication)
 - URL: permet l'accès à des objets distants
 - RMI: Remote Method Invocation: qui permet d'appeler des méthodes distantes

1. Introduction à Java

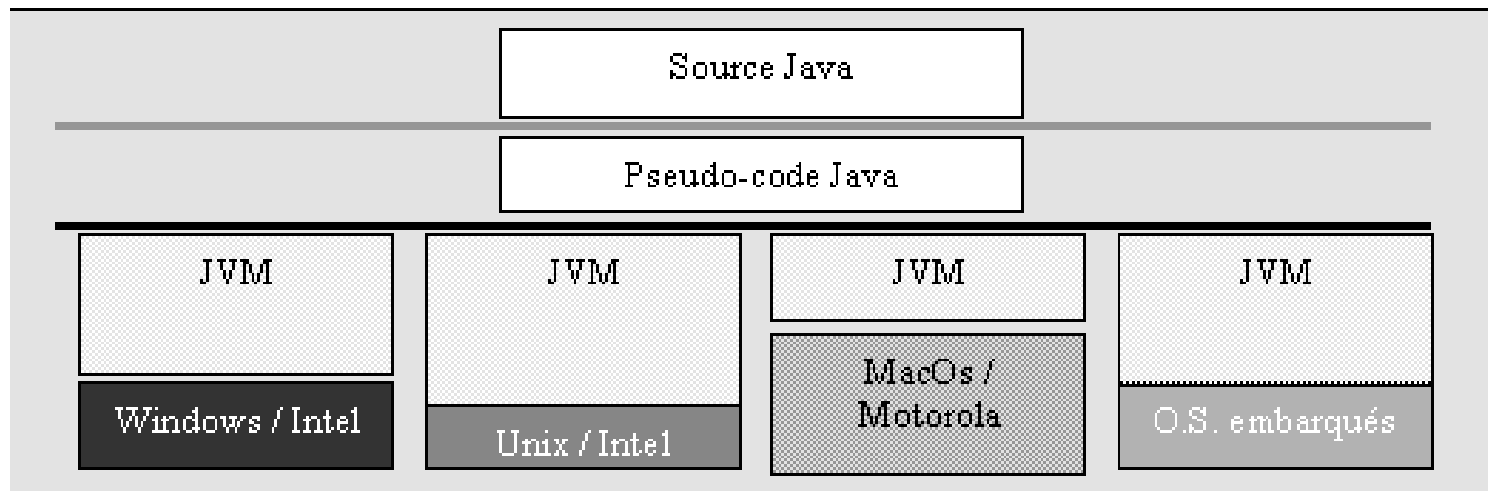
- Le langage Java est un langage intermédiaire:
 - Un programme Java **n'est pas compilé en code machine** ;
 - Il sera **compilé** en code intermédiaire interprété nommé **ByteCode**.
 - Lors de l'exécution le **ByteCode** sera interprété à l'aide d'une machine dite virtuelle **JVM** (Java Virtual Machine).
- Le langage Java est portable et indépendant des plates-formes :
 - Le code intermédiaire produit «**ByteCode**» est indépendant des plates-formes.
 - Il pourra être exécuté sur tous types de machines et systèmes possédant l'interpréteur de code Java « JVM ».

1. Introduction à Java

- **Le langage Java est robuste et sûr :**
 - Détection des erreurs d'invocation des méthodes :
 - java est fortement typé
 - seules des conversions sûres sont automatiques
- **Fiabilité de la gestion de la mémoire:**
 - pas de pointeurs
 - ramasse miettes automatique
 - contrôle automatique d'accès aux tableaux et aux chaînes de caractères
- **Le langage Java est multithread :**
 - JAVA permet l'exécution simultanée de plusieurs processus légers (thread)
 - Classe **java.lang.Thread** avec les méthodes permettant de :
 - Démarrer, Exécuter, Stopper ces processus.
 - contrôler les synchronisations et l'état cohérent des données.

1. Introduction à Java

- Java se contente de :
 - proposer des **API** universelles (**A**pplication **P**rogramming **I**nterface)
 - fournir un compilateur en "pseudo-code" universel.
- Les fournisseurs de système devront quant à eux fournir la "machine virtuelle"

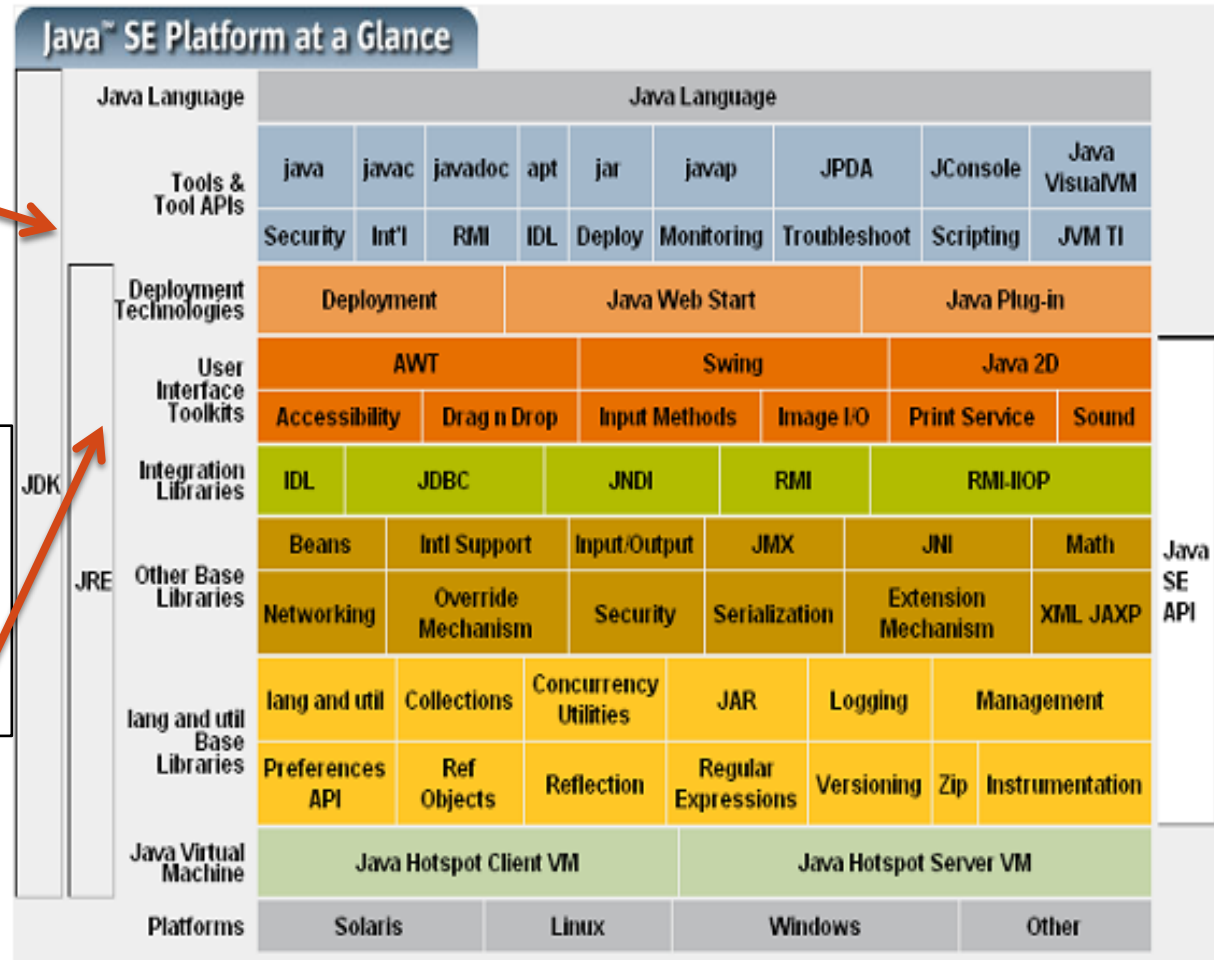


1. Introduction à Java

- **Un langage complet:**
 - **Structures** et **Ressources** couvrent TOUS les besoins qui permettent la programmation d'une application complète;

JDK (Java Development Kit) qui contient une version complète de Java (**machine virtuelle** + **compilateur** + **utilitaires de développement**).

JRE (Java Runtime Environment) contient une **machine virtuelle** Java + quelques **utilitaires d'exploitation**. Cette distribution ne permet pas de développer.



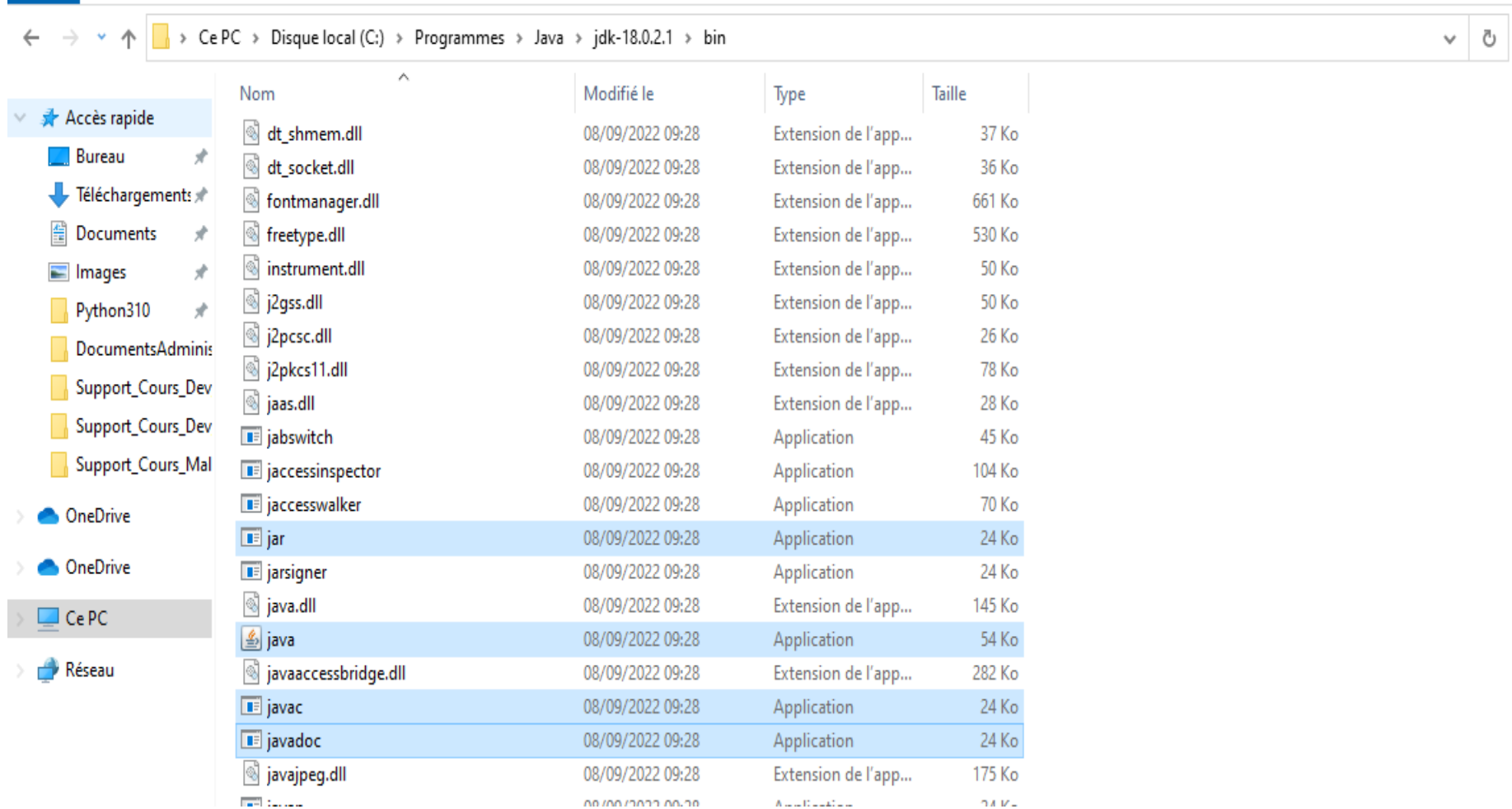
2. Mise en œuvre

- **Analyse contenu et répertoires du jdk**

Nom	Modifié le	Type	Taille
bin	08/09/2022 09:28	Dossier de fichiers	
conf	08/09/2022 09:28	Dossier de fichiers	
include	08/09/2022 09:28	Dossier de fichiers	
jmods	08/09/2022 09:28	Dossier de fichiers	
legal	08/09/2022 09:28	Dossier de fichiers	
lib	08/09/2022 09:28	Dossier de fichiers	
COPYRIGHT	16/08/2022 00:59	Fichier	4 Ko
LICENSE	08/09/2022 09:28	Fichier	7 Ko
README	08/09/2022 09:28	Fichier	1 Ko
release	08/09/2022 09:28	Fichier	2 Ko

2. Mise en œuvre

- **Analyse du contenu du répertoire 'bin'**



Nom	Modifié le	Type	Taille
dt_shmem.dll	08/09/2022 09:28	Extension de l'app...	37 Ko
dt_socket.dll	08/09/2022 09:28	Extension de l'app...	36 Ko
fontmanager.dll	08/09/2022 09:28	Extension de l'app...	661 Ko
freetype.dll	08/09/2022 09:28	Extension de l'app...	530 Ko
instrument.dll	08/09/2022 09:28	Extension de l'app...	50 Ko
j2gss.dll	08/09/2022 09:28	Extension de l'app...	50 Ko
j2pccs.dll	08/09/2022 09:28	Extension de l'app...	26 Ko
j2pkcs11.dll	08/09/2022 09:28	Extension de l'app...	78 Ko
jaas.dll	08/09/2022 09:28	Extension de l'app...	28 Ko
jabswitch	08/09/2022 09:28	Application	45 Ko
jaccessinspector	08/09/2022 09:28	Application	104 Ko
jaccesswalker	08/09/2022 09:28	Application	70 Ko
jar	08/09/2022 09:28	Application	24 Ko
jarsigner	08/09/2022 09:28	Application	24 Ko
java.dll	08/09/2022 09:28	Extension de l'app...	145 Ko
java	08/09/2022 09:28	Application	54 Ko
javaaccessbridge.dll	08/09/2022 09:28	Extension de l'app...	282 Ko
javac	08/09/2022 09:28	Application	24 Ko
javadoc	08/09/2022 09:28	Application	24 Ko
javajpeg.dll	08/09/2022 09:28	Extension de l'app...	175 Ko

2. Mise en œuvre

- **Le JDK**

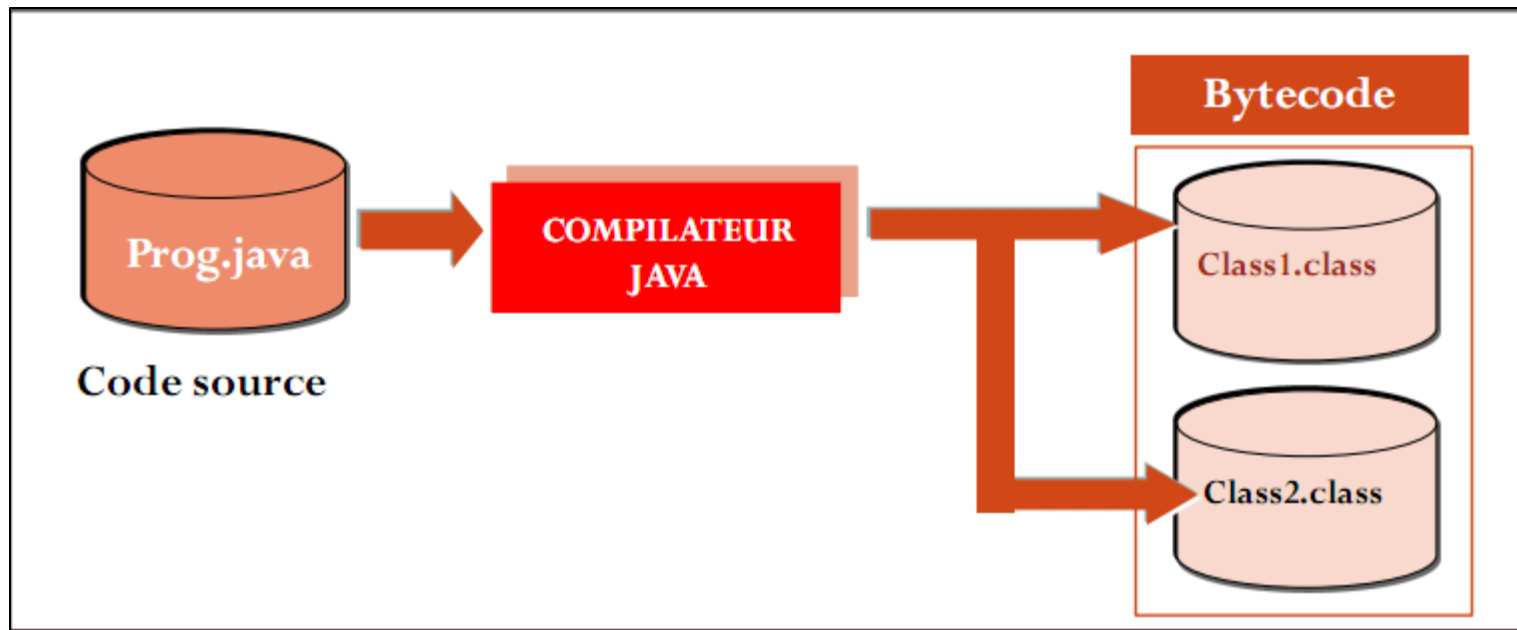
- Java fournit un kit de développement (**S**oftware **D**evelopment **K**it) :
La production de programmes Java. Ce kit contient essentiellement:

- Un compilateur: **javac (JavaCompiler)**
- Une machine virtuelle pour lancer les programmes: **java**
- Un utilitaire de documentation: **javadoc**
- Des utilitaires divers de compression, de gestion du modèle de sécurité, de débogage, etc.
- Les librairies essentielles (paquets standards) de Java.

2. Mise en œuvre

1

Production d'une application Java standard



Bytecode = Pseudo code machine qui contrairement au code binaire natif, n'est pas exécuté directement par le processeur



ce qui a pour conséquence de ralentir son exécution.

2. Mise en œuvre

2

Exécution d'une application Java standard



JVM = Programme capable d'interpréter les instructions contenues dans les fichiers **ByteCode** Java afin de les exécuter.

3. Java par l'exemple

```
Fichier «XXX.java »  
// Commentaire  
Définition d'une classe XXX  
Méthode main ( )
```

- Une application Java minimale doit contenir une classe :
 - Portant le même nom que le fichier ayant l'extension « .java » dans le quel elle est enregistré.
 - Comportant (au moins) une méthode :
 - appelée **main**,
 - de type **public** et **static**,
 - ayant un argument de type **String[]**,

3. Java par l'exemple

Fichier «PremierProgramme.java »

```
// Ce programme se contente d'afficher le message « Ca Marche »
```

- La première ligne du programme «PremierProgramme» est une ligne de commentaire:
 - Elle commence par //.
 - Tout ce qui est compris entre ces deux caractères et la fin de la ligne est ignoré par le compilateur.
 - En revanche, d'autres outils peuvent utiliser le texte contenu dans ce type de commentaire.

3. Java par l'exemple

```
class PremierProgramme  
{  
  
}
```

- Le mot **class** veut dire que nous allons définir une **nouvelle classe Java**, suivi du nom de cette classe.
- En Java, les majuscules et les minuscules sont considérés comme des caractères différents. **PremierProgramme** **n'est pas identique** à **PREMIERProgramme**.
- Les caractères «**{**» et «**}**» marquent le début et la fin du bloc d'instructions à réaliser par la classe.

3. Java par l'exemple

```
public static void main(String args[] )  
{  
    System.out.println("Ca Marche");  
}
```

- **main()** : signifie que nous allons définir une méthode appelée main.
- Le mot **main** indique que cette méthode est la méthode principale de la classe.
- Un **interpréteur Java** a pour fonction **d'exécuter les instructions** contenus dans le bloc d'instruction de la méthode principale main, du programme qu'on lui soumet.
- Une **méthode** est une sorte de **procédure** (ensemble d'instructions) **appartenant à une classe**.

3. Java par l'exemple

```
public static void main(String args[] )  
{  
    System.out.println("Ca Marche");  
}
```

- Une méthode peut prendre des paramètres de types précis et renvoie éventuellement une valeur de type tout aussi précis.
- Le mot **void** signifie que la méthode `main` ne renvoie aucune valeur.
- **args[]** est le paramètre d'entrée de type `String` de la méthode `main`.
- Les mots **public** et **static** décrivent chacun une caractéristique de la méthode (voir plus loin).
- **System.out.println** est une commande permettant d'afficher la chaîne de caractère «Ca MARCHE» sur la sortie par défaut de votre machine qui est l'écran.

4. Structure d'une application Java standard

Fichier «XXX.java »

// Commentaire

Définition d'une classe AAA

// Commentaire

Définition d'une classe BBB

⋮

Définition d'une classe XXX

Méthode main ()



Parmi les classes définies dans le fichier XXX.java il ne peut y avoir qu'une seule classe publique et cette classe doit avoir le même nom XXX que le fichier.

5. Écrire une application Java avec un IDE

- **IDE:** Integrated Development Environment (Environnement de développement intégré).

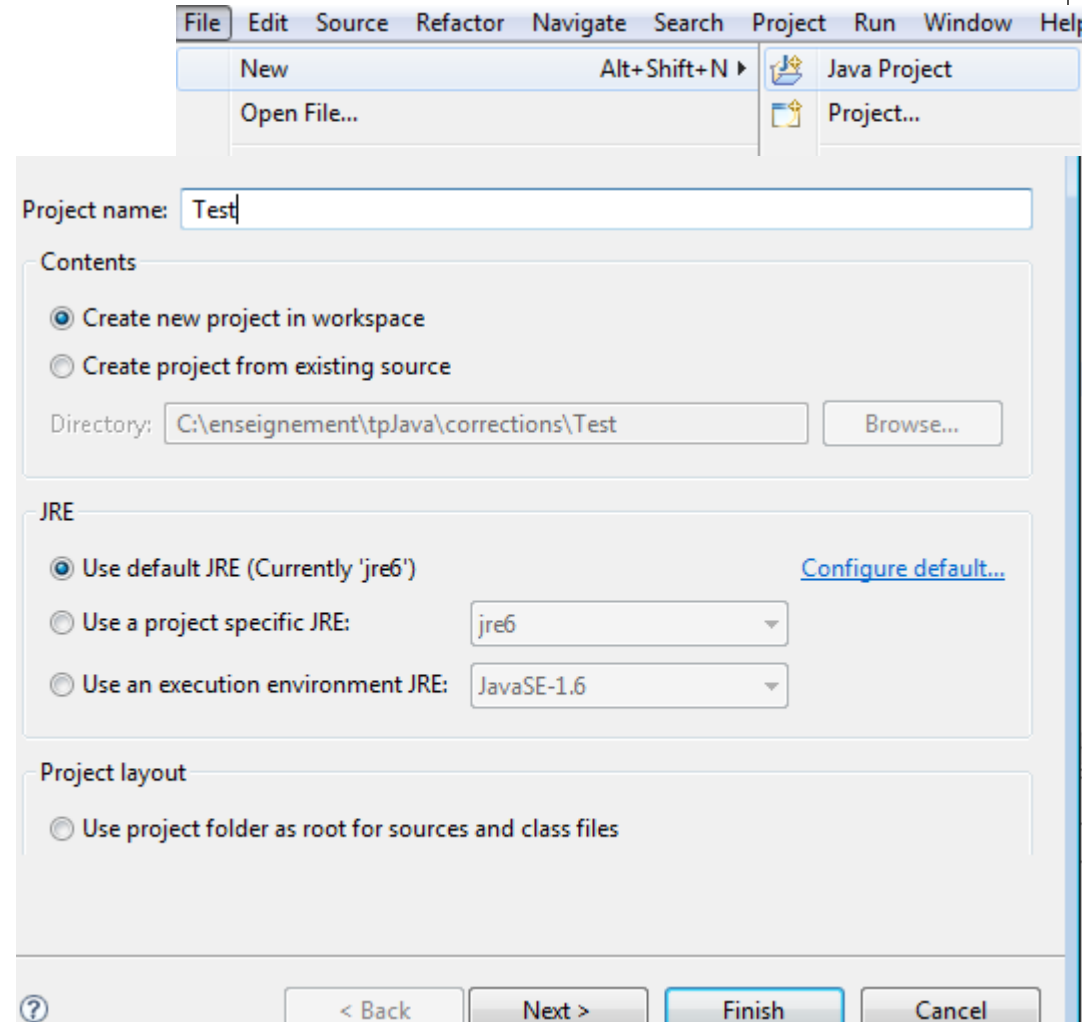
- **Pré requis**

- ECLIPSE: www.eclipse.org



- **Étapes**

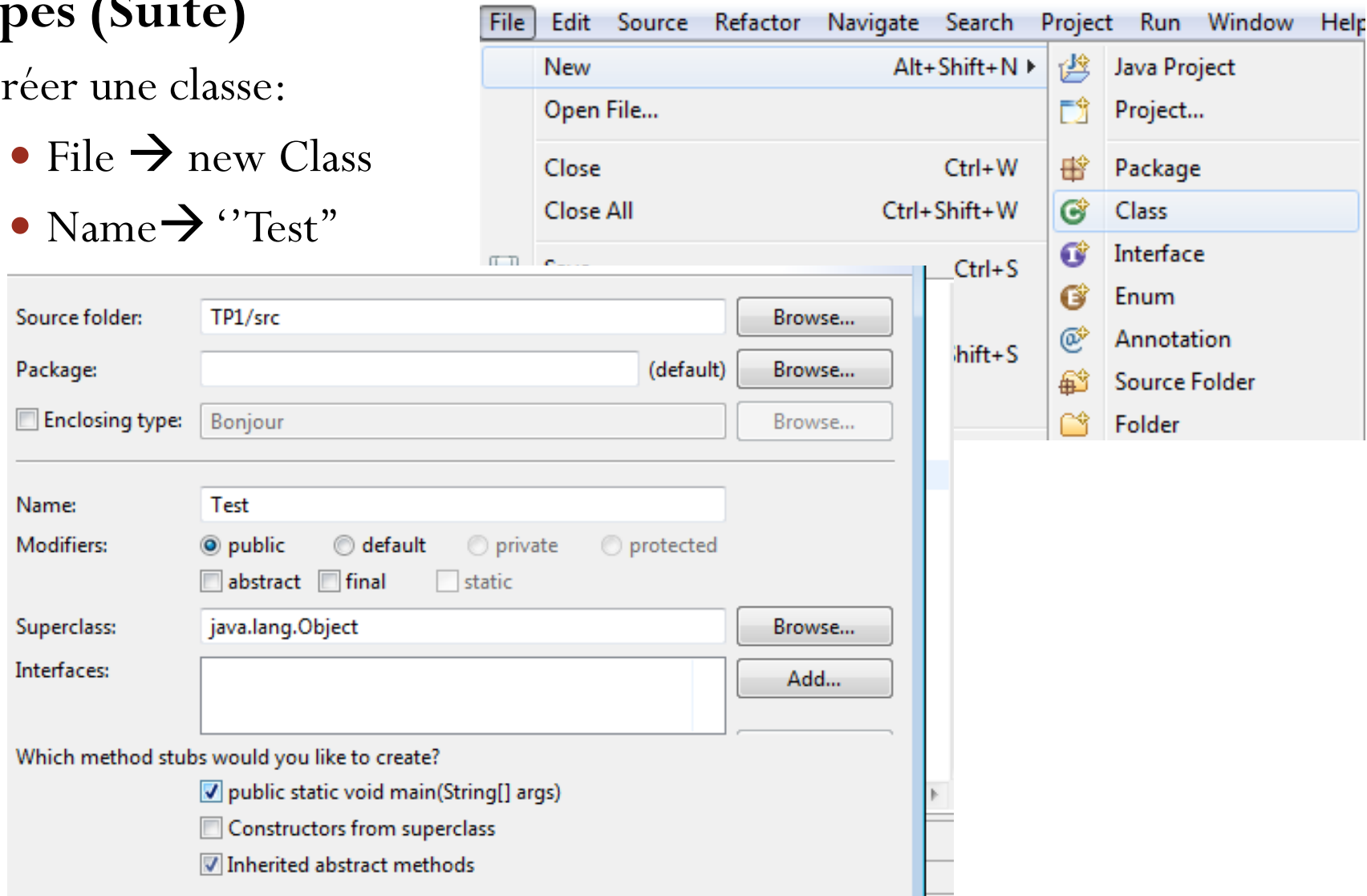
- Créer un projet:
 - File → new Java Project
 - Project name → "Test"



5. Écrire une application Java avec un IDE

- Étapes (Suite)

- Créer une classe:
 - File → new Class
 - Name → "Test"



5. Écrire une application Java avec un IDE

- Étapes (Suite)
 - Insérer l'instruction « `System.out.println("Ca Marche");` » dans le corps de la méthode main.

```
public class Test {  
    /**  
     * @param args  
     */  
    public static void main(String[] args) {  
        System.out.println("ça marche");  
    }  
}
```

Un programme Java complet

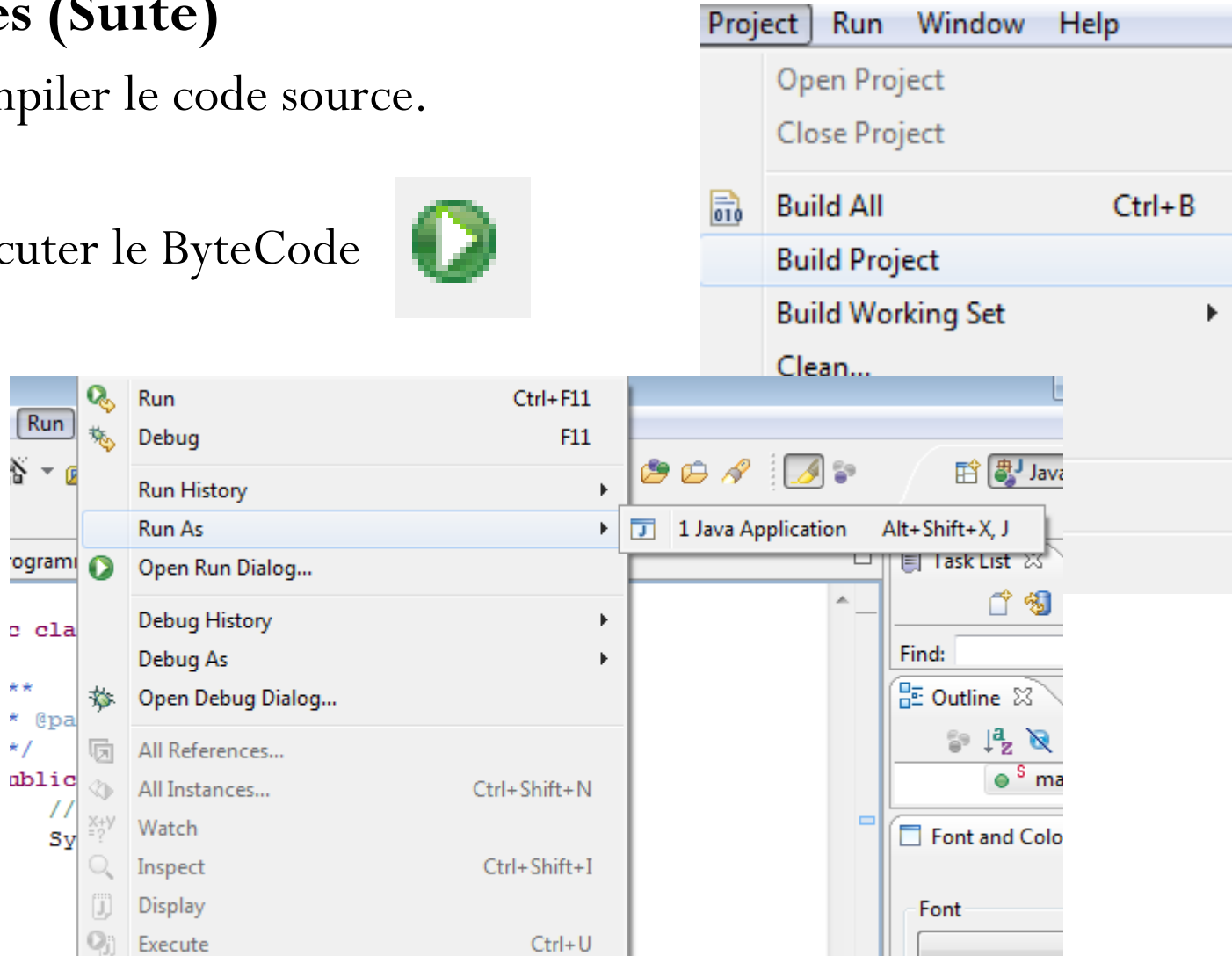
- `public static void main(String args[])`
 - La méthode **main** est le **point d'entrée** du programme, elle est :
 - **public** : car elle est appelée de l'extérieur de la classe par la **JVM**.
 - **static** : car elle n'a pas besoin d'un objet pour être invoquée. Elle est appelée par le **JVM** avant toute création d'instance (elle est une méthode de classe).
 - **void** : car elle ne renvoie pas de valeur.
 - La méthode **main** a comme argument un tableau de **String** ce qui lui permet de récupérer des paramètres introduits sur la ligne de commande (comme en C++).
 - la commande `C:\> java MonProgramme param1 param2 param3` lance l'exécution de la classe **MonProgramme** avec comme paramètres les chaînes **param1** , **param2**, **param3** dans `args[0]`, `args[1]` et `args[2]`.

5. Écrire une application Java avec un IDE

- Étapes (Suite)

- Compiler le code source.

- Exécuter le ByteCode



6. Syntaxe de base : Notion de type

- La mémoire centrale est un **ensemble de bits** regroupés **en octets**.
- Chaque **octet** est **repéré** par **son adresse**
- Pour **stocker** une information dans la mémoire :
 - Savoir le **nombre d'octet** nécessaire
 - Un **codage approprié**
- La notion de **type** sert à régler ces problèmes.
- Les **types primitifs** de Java se répartissent en quatre catégories:
 - Nombres **entiers**
 - Nombres **flottants**
 - **Caractères**
 - **booléens**

6. Syntaxe de base : Les types entiers

Il existe 4 types entiers dans Java:

Type	Taille (octets)	Valeur minimale	Valeur maximale
byte	1	-128 (Byte.MIN_VALUE)	127 (Byte.MAX_VALUE)
short	2	-32 768 (Short.MIN_VALUE)	32 767 (Short.MAX_VALUE)
int	4	-2 147 483 648 (Integer.MIN_VALUE)	2 147 483 647 (Integer.MAX_VALUE)
long	8	-9 223 372 036 854 775 808 (Long.MIN_VALUE)	9 223 372 036 854 775 807 (Long.MAX_VALUE)

6. Syntaxe de base : Les types entiers

- Les variables peuvent être déclarées n'importe où dans un bloc.
- Essayer d'écrire ce code sur la machine est faite la compilation:

```
int i=258;
```

```
long l=i; //ok
```

```
byte b=i; // error: Explicit cast needed to  
// convert int to byte
```

```
byte b=258; //error: Explicit cast needed  
// to convert int to byte
```

```
byte b=(byte) i; //ok mais b=2
```

6. Syntaxe de base : Les types flottants

- Un nombre réel sera représenté en **flottant** en déterminant un signe s (+1 ou -1) et deux quantités M (mantisse) et E (exposant) telle que la valeur: $sM.2^E$
- Java prévoit deux types de flottants: **float** (réel avec simple précision) et **double** (réel avec double précision)

Type	Taille (octets)	Précision	Valeur absolue minimale	Valeur absolue maximale
float	4	7	$-1.40239846 E^{-45}$ (Float.MIN_VALUE)	$3.40282347 E^{38}$ (Float.Max_VALUE)
double	8	15	$4.9406564584124654E^{-324}$ (Double.MIN_VALUE)	$1.797693134862316 E^{308}$ (Double.MAX_VALUE)

6. Syntaxe de base : Les types flottants

- Exemple d'initialisation des variables flottantes:

```
float x;
```

```
x=12.5;
```

Erreur de compilation

➔ Toutes les valeurs constantes **flottantes** sont créés par le **compilateur** dans le type **double**.

Solution:

```
float x;
```

```
x=12.5f;
```

6. Syntaxe de base : Le type caractère

- En langage C, le type caractère est représenté sur 1 octet.
- En Java, le type caractère (`char`) est représenté sur 2 octets.
- Pour initialiser une variable de type caractère avec une constante, on doit utilisé les ' '.
- **Exemple:**
 - `char c = 'A' ;`

6. Syntaxe de base : Le type booléen

- Ce type sert à représenter une valeur logique du type *true/false*.
- Il n'existe pas dans tous les langages, tel que le langage C.
- **Syntaxe:**
 - *boolean b;*
- **Exemple d'initialisation:**
 - *b=true; b= 1<2;*

6. Syntaxe de base : Déclaration de constantes

- Les variables se déclarent comme dans les autres langages

```
type nomVariable = valeur
```

- Les constantes sont déclarées grâce au mot-clé **final**

```
final type nomVariable = valeur
```

- Il existe également des constantes de classes (à voir prochainement)

```
static final type nomVariable= valeur
```

- **Exemple :**

```
final int n = 10;
```

6. Syntaxe de base : Les opérateurs

- Opérateur d'affectation

=

- Opérateurs arithmétiques

+ - * / % ++ -- += -= *= /= %=

- Opérateurs relationnels

> >= == != < <=

- Opérateurs logiques

&& || !

- Opérateurs binaires

& | ^ ~ >> << >>>

6. Syntaxe de base : Les opérateurs

- L'opérateur logique ?
 - composé de trois expressions
 - **expression1 ? expression2 : expression3**
 - si **expression1** vraie alors **expression2**
 - sinon **expression3**
 - `val = (x >= 0) ? x : -x;`
- Opérateur d'allocation mémoire : **new**

6. Syntaxe de base : Structures de contrôle

- Essentiellement les mêmes qu'en C
 - if, switch, for, while, do while
 - ++, +=, &&, &, <<, ?:
- Plus les blocs labellisés (style Pascal):
 - Les instructions de branchement inconditionnel `break` et `continue`

6. Syntaxe de base : Structures de contrôle

- L'instruction **break** ordinaire permet de sortir
 - d'un switch
 - d'un for
 - d'un while
 - d'un do

```
for (;;)
{
    if (EVENEMENT) break;
    /* si EVENEMENT devient VRAI on sort */
}
```

6. Syntaxe de base : Structures de contrôle

- L'instruction **continue** ordinaire permet de passer prématurément au tour de boucle suivant.
- Exemple avec for :

```
public class Cont1
{ public static void main(String [] args) {
    int i;
    for(i=1;i<=5;i++)
    {   System.out.println("début tour "+i);
        if (i<4) continue;
        System.out.println("fin tour "+i);
    }
    System.out.println("après la boucle");
}
```

```
début tour 1
début tour 2
début tour 3
début tour 4
fin tour 4
début tour 5
fin tour 5
après la boucle
```

6. Syntaxe de base : Les Tableaux

- **Déclaration :**

```
int [ ] unTableauDEntiers ; // équivalent à : int unTableauDEntiers[];  
long[ ][ ] unTableau2D ;
```

- **Création et initialisation :**

```
unTableauDEntiers= new int [20];  
unTableau2D = new long[256][256];  
unAutreTableauDEntiers= new int[2*size];  
int[] primes = {1, 2, 3, 5, 7, 7+4};
```

- **Utilisation :**

```
int L = unTableauDEntiers.length; // L = 42  
int e = unTableauDEntiers[30]; // Une exception : 30 indice inexistant
```

6. Syntaxe de base : Les Tableaux multidimensionnels

- Un tableau multidimensionnel est un tableau de tableaux.

```
int[ ][ ] t = new int[5][ ];  
  
for (int i=0; i<t.length; i++)  
{  
    t[i]=new int[5-i];  
}  
  
for (int i=0; i<t.length; i++)  
{  
    for (int j=0; j<t[i].length; j++)  
    {  
        t[i][j]=i+j;  
    }  
}
```

	[0]	[1]	[2]	[3]	[4]
t[0]	0	1	2	3	4
t[1]	1	2	3	4	
t[2]	2	3	4		
t[3]	3	4			
t[4]	4				

6. Syntaxe de base : La classe String

- `String ch="bonjour" ;`
 - longueur : `int n = ch.length()` ;
 - accès au ième caractère : `c = ch.charAt(0)`;
 - remplacement de caractère : `ch.replace('o', 'a') // banjaour`
 - concatenation : `String ch2 = ch + ch`;
 - recherche d'un caractère dans 1 chaîne : `n = ch.indexOf('o') // n vaut 1`
 - recherche de sous-chaîne : `n = ch.indexOf("jo") // n vaut 3`
 - dernière occurrence : `lastIndexOf()`
 - extraction de sous-chaîne : `ch2 = ch.substring(2) ; // ch2 = njaour`
 - comparaison de 2 chaînes : `compareTo()`
 - Majuscules et minuscules : `toLowerCase()`; `toUpperCase()`;
 - opérateur `==` et `!=` travaillent sur les **références des objets**, pour tester l'égalité utilisez la fonction `equals`.

6. Syntaxe de base : les conversions des types

- Il y a 2 catégories de conversions possibles :
 - Conversions **explicites**:
 - Celles faites sur une demande explicite par un **programmeur**.
 - Conversions **implicites**:
 - Celles faites automatiquement par un **compilateur**:
 - Lors d'une **affectation**,
 - Lors d'une **promotion arithmétique**,
 - Lors d'un **passage de paramètres** (lors de l'invocation d'une méthode),

6. Syntaxe de base : les conversions des types

- **Conversion explicite:**
 - Objectif:
 - **Changer** le type d'une donnée si **besoin**.
 - Comment?
 - Préfixer l'opérande par le type choisi.
 - Encadrer le type choisi par des parenthèses.
- Exemple:
 - `double d=2.5;`
 - `long l=(long) d;`

6. Syntaxe de base : les conversions des types

- **Conversion implicite lors d'une affectation:**
 - Objectif:
 - Changer **automatiquement** le type d'une donnée si besoin.
 - Comment?
 - Exemple1:
 - Contrairement à ce qui se passe dans C, les conversions **pouvant provoquer une perte de valeur** ne sont pas autorisées de manière implicite dans C: cela doit être explicitement demandé.
 - Exemple2:
 - Un type char peut être utilisé par tout où une valeur de type int est permis.

6. Syntaxe de base : les conversions des types

- **Conversion implicite lors d'une affectation :**
 - Illustration pour l'exemple 1:

```
public class ProgrammeAffectation {  
    public static void main(String[ ] args)  
    {  
        int i;  
        short j = 2 ;  
        i = j ;  
  
        float k= 1.2 f ;  
        i = k;  
    }  
}
```

Conversion
implicite

Erreur de
Compilation

- Solution:

```
i = (int) k;
```

6. Syntaxe de base : les conversions des types

- Conversion implicite lors d'une opération arithmétique:
 - Illustration pour l'exemple 2:

```
public class ProgrammeAffectation {  
    public static void main(String[] args)  
    {  
        int i;  
        i = 'A';  
  
        System.out.println(i);  
    }  
}
```



Conversion implicite

- Résultat d'affichage:

65

6. Syntaxe de base : les conversions des types

- **Conversion implicite lors d'une opération arithmétique:**
 - Objectif :
 - Si un opérateur s'applique sur deux arguments de type différent un des deux arguments sera converti dans le type de l'autre.
 - Exemple :

```
public class ProgrammeAffectation {  
    public static void main(String[] args)  
    {  
        int i;  
        i = 'A' - 1;  
  
        System.out.println(i);  
    }  
}
```

Conversion
implicite



- Résultat d'affichage:

64

6. Syntaxe de base : les conversions des types

- Conversion implicite lors d'un passage de paramètres (lors de l'invocation d'une méthode) :

```
public class ProgrammeSoustraction {
```

```
    static short entierShort;
```

```
    static void Afficher( int entier )
```

```
    {
```

```
        System.out.print( entier ) ;
```

```
    }
```

```
    public static void main(String[ ] args)
```

```
    {
```

```
        entierShort = 2 ;
```

```
        Afficher(entierShort )
```

```
    }
```

```
}
```

Invocation de la
méthode Afficher

6. Syntaxe de base : les conversions des types

- **Attention :**
 - Il n'y a pas de conversion possible (implicite ou explicite) entre un type **entier** et le type **boolean** :

The diagram shows a code snippet with two `if` statements. The first `if (i)` is marked with a red arrow pointing to a box labeled "Erreur de Compilation". The second `if (i != 0)` is marked with a red arrow pointing to a box labeled "OK".

```
int i = 0;

if (i)
{
    .....
}

if (i != 0)
{
    .....
}
```

7. Références Utiles

- <https://docs.oracle.com/en/java/javase/18/>
- <https://www.eclipse.org/>
- <https://java.developpez.com/>